SEQUENTIAL LOGIC CIRCUIT

1) **Sequential logic** is a type of logic circuit whose output depends not only on the present value of its input signals but on the sequence of past inputs.

This is in contrast to *combinational logic*, whose output is a function of only the present input. That is, sequential logic has *state* (*memory*) while combinational logic does not.

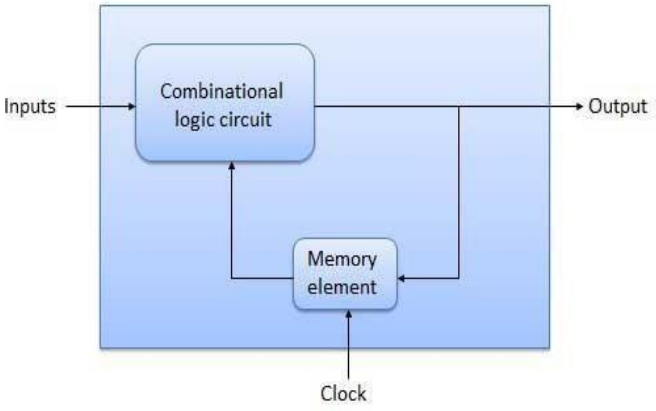
2) Sequential circuit has memory so output can vary based on input. This type of circuits uses previous input, output, clock and a memory element.

The combinational circuit does not use any memory. Hence the previous state of input does not have any effect on the present state of the circuit.

3) Sequential circuits use current input variables and previous input variables by storing the information and putting back into the circuit on the next clock (activation) cycle.

Combinational circuits and systems produce an output based on input variables only.

Block diagram



Flip Flop

A **flip-flop** or **latch** is a circuit that has two stable states and can be used to store state information. Flip-flops and latches are used as data storage elements.

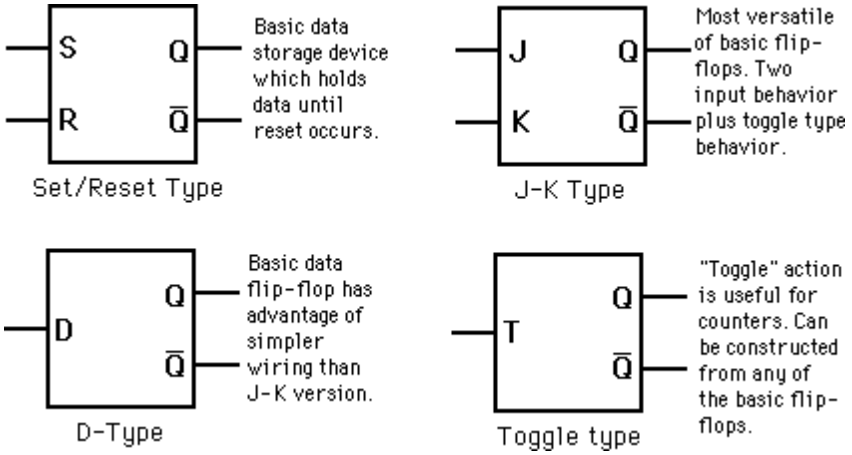
A flip-flop stores a single *bit* (binary digit) of data; one of its two states represents a "one" and the other represents a "zero". Such data storage can be used for storage of *state*, and such a circuit is described as sequential logic.

Flip-flops can be either simple (transparent or opaque) or clocked (synchronous or edge-triggered).

The most commonly used application of flip flops is in the implementation of a feedback circuit.

There are mainly four types of flip flops that are used in electronic circuits. They are-

1. **The basic Flip Flop or S-R Flip Flop**
2. **Delay Flip Flop [D Flip Flop]**
3. **J-K Flip Flop**
4. **T Flip Flop**



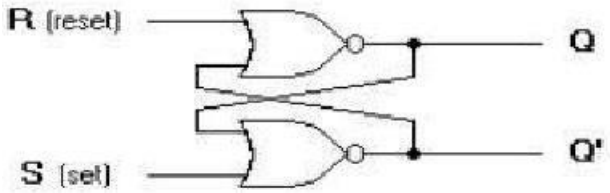
1. S-R Flip Flop

The SET-RESET flip flop is designed with the help of two NOR gates and also two NAND gates. These flip flops are also called S-R Latch.

- ***S-R Flip Flop using NOR Gate***

The design of such a flip flop includes two inputs, called the SET [S] and RESET [R]. There are also two outputs, Q and Q'.

The diagram and truth table is shown below-



(a) Logic diagram

S	R	Q	Q'	Result
1	0	1	0	SET
0	0	1	0	NO CHANGE
0	1	0	1	RESET
0	0	0	1	NO CHANGE
1	1	0	0	FORBIDDEN

(b) Truth table

From the diagram it is evident that the flip flop has mainly four states. They are

S=1, R=0—Q=1, Q'=0
This state is also called the SET state.

S=0, R=1—Q=0, Q'=1
This state is known as the RESET state.

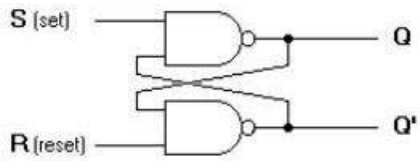
In both the states you can see that the outputs are just compliments of each other and that the value of Q follows the value of S.

S=0, R=0—Q & Q' = Remember
If both the values of S and R are switched to 0, then the circuit remembers the value of S and R in their previous state.

S=1, R=1—Q=0, Q'=0 [Invalid/forbidden]
This is an invalid state because the values of both Q and Q' are 0. They are supposed to be complements of each other. Normally, this state must be avoided.

- ***S-R Flip Flop using NAND Gate***

The circuit of the S-R flip flop using NAND Gate and its truth table is shown below.



(a) Logic diagram

S	R	Q	Q'	Result
1	0	1	0	SET
0	0	1	0	NO CHANGE
0	1	0	1	RESET
0	0	0	1	NO CHANGE
1	1	0	0	FORBIDDEN

(b) Truth table

Like the NOR Gate S-R flip flop, this one also has four states. They are

S=1, R=0—Q=0, Q'=1
This state is also called the SET state.

S=0, R=1—Q=1, Q'=0
This state is known as the RESET state.

In both the states you can see that the outputs are just compliments of each other and that the value of Q follows the compliment value of S.

S=0, R=0—Q=1, & Q' =1 [Invalid/forbidden]
If both the values of S and R are switched to 0 it is an invalid state because the values of both Q and Q' are 1. They are supposed to be complements of each other. Normally, this state must be avoided.

S=1, R=1—Q & Q'= Remember
If both the values of S and R are switched to 1, then the circuit remembers the value of S and R in their previous state.

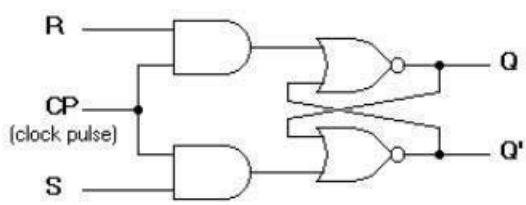
- ***Clocked S-R Flip Flop***

It is also called a Gated S-R flip flop.

The problems with S-R flip flops using NOR and NAND gate is the invalid/forbidden state. This problem can be overcome by using a bistable SR flip-flop that can change outputs when certain invalid states are met, regardless of the condition of either the Set or the Reset inputs.

For this, a clocked S-R flip flop is designed by adding two AND gates to a basic NOR Gate flip flop.

The circuit diagram and truth table is shown below.



(a) Logic diagram

clock	S	R	Q	$\bar{Q}$	STATUS
↑	0	0	Q	$\bar{Q}$	HOLD (NoChange)
↑	0	1	0	1	RESET
↑	1	0	1	0	SET
↑	1	1	0	0	INVALID

b) Truth Table

A clock pulse [CP] is given to the inputs of the AND Gate. When the value of the clock pulse is '0', the outputs of both the AND Gates remain '0'. As soon as a pulse is given the value of CP turns '1'. This makes the values at S and R to pass through the NOR Gate flip flop.

When the values of both S and R values turn '1', the HIGH value of CP causes both of them to turn to '0' for a short moment. As soon as the pulse is removed, the flip flop state becomes invalid/forbidden.

2. <u>D Flip Flop</u>

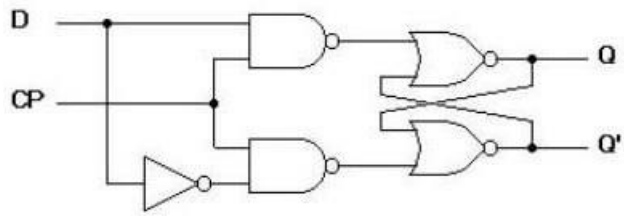
Delay Flip Flop / D Flip Flop

Delay Flip Flop or D Flip Flop is the simple gated S-R latch with a NAND inverter connected between S and R inputs. It has only one input.

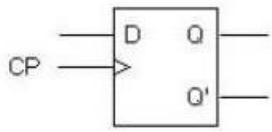
The input data is appearing at the output after some time. Due to this data delay between i/p and o/p, it is called delay flip flop.

S and R will be the complements of each other due to NAND inverter. Hence S = R = 0 or S = R = 1, these input condition will never appear. This problem is avoid by SR = 00 and SR = 1 conditions.

The circuit diagram and truth table is given below.



(a) Logic diagram with NAND gates



(b) Graphical symbol

Inputs		Outputs		
D	CLK	*Q*	$\overline{Q}$	Comments
1	↑	1	0	SET
0	↑	0	1	RESET

c) Truth Table

D flip flop is actually a slight modification of the clocked SR flip-flop. From the figure we can see that the D input is connected to the S(SET) input and the complement of the D input is connected to the R(RESET) input.

The D input is passed on to the flip flop when the value of CP is '1'. When CP is HIGH, the flip flop moves to the SET state. If it is '0', the flip flop switches to the CLEAR/RESET state.

3. J-K Flip Flop

A J-K flip flop can also be defined as a modification of the S-R flip flop. The only difference is that the intermediate state is more refined and precise than that of a S-R flip flop.

J-K means Jack Kilby , an engineer who invented IC. The behavior of inputs J and K is same as the S and R inputs of the S-R flip flop. The letter J stands for SET and the letter K stands for RESET.

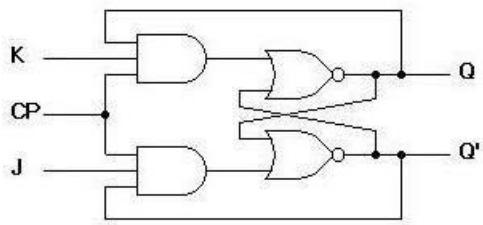
When both the inputs J and K have a HIGH state, the flip-flop switch to the toggle(complement) state. So, for a value of Q = 1, it switches to Q=0 and for a value of Q = 0, it switches to Q=1.
The circuit includes two 3-input AND gates. The output Q of the flip flop is returned back as a feedback to the input of the AND along with other inputs like K and clock pulse [CP].

So, if the value of CP is '1', the flip flop gets a CLEAR signal and with the condition that the value of Q was earlier 1. Similarly output Q' of the flip flop is given as a feedback to the input of the AND along with other inputs like J and clock pulse [CP]. So the output becomes SET when the value of CP is 1 only if the value of Q' was earlier 1.

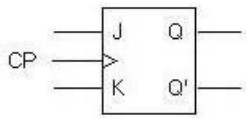
When the input to the JK flip-flop is J=1 and K=1, the **Race around condition** occurs i.e. if the width of the clock pulse C_P is too long, the state of the flip-flop will keep on changing from 0 to 1, 1 to 0 , 0 to 1 and so on, and at the end of the clock pulse, its state will be uncertain.

To avoid race around condition we use- Master slave JK flip flop

The circuit diagram and truth-table of a J-K flip flop is shown below.



(a) Logic diagram



(b) Graphical symbol

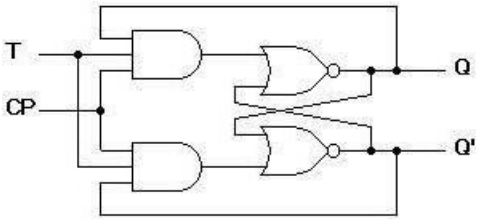
J	K	C	Q	$\overline{Q}$
0	0	↑	no change	
0	1	↑	0	1
1	0	↑	1	0
1	1	↑	toggle	

c) Truth Table

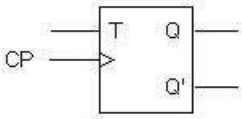
4. T Flip Flop

This is a much simpler version of the J-K flip flop. Both the J and K inputs are connected together and thus are also called a single input J-K flip flop.

When clock pulse is given to the flip flop, the output begins to toggle.



(a) Logic diagram



(b) Graphical symbol

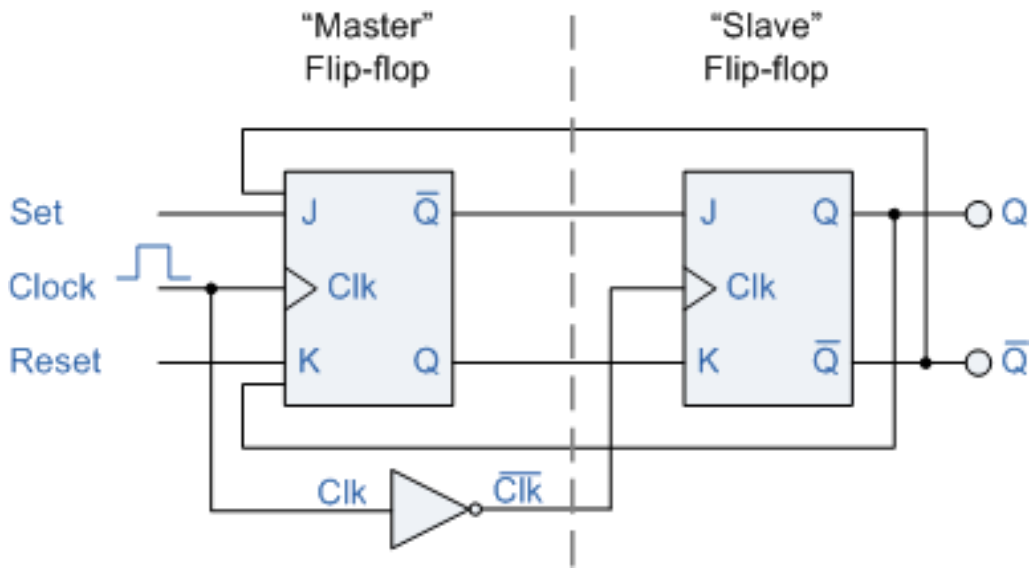
T	*CLK*	*Q(t+1)*	Comments
0	↑	*Q(t)*	No change
1	↑	*Q(t)'*	Toggle

c) Truth Table

Operation

S.N.	Condition	Operation
1	T = 0, J = K = 0	The output Q and Q bar won't change
2	T = 1, J = K = 1	Output will toggle corresponding to every leading edge of clock signal.

5. Master-Slave Flip Flop



Master-slave flip flop is designed using two separate flip flops. Out of these, one acts as the master and the other as a slave.

From the above figure we can see that both the J-K flip flops are presented in a series connection. The output of the master J-K flip flop is fed to the input of the slave J-K flip flop.

The output of the slave J-K flip flop is given as a feedback to the input of the master J-K flip flop.

The clock pulse [Clk] is given to the master J-K flip flop and it is sent through a NOT Gate and thus inverted before passing it to the slave J-K flip flop.

Operation

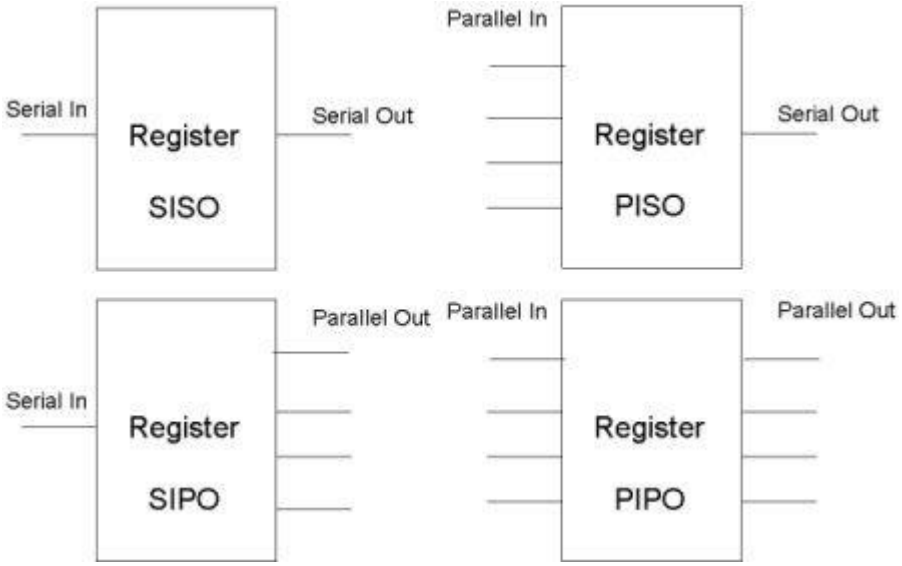
S.N.	Condition	Operation
1	**J = K = 0 (No change)**	When clock = 0, the slave becomes active and master is inactive. But since the S and R inputs have not changed, the slave outputs will also remain unchanged. Therefore outputs will not change if J = K =0.
2	**J = 0 and K = 1 (Reset)**	Clock = 1 − Master active, slave inactive. Therefore outputs of the master become Q = 0 and Q' = 1. That means S = 0 and R =1. Clock = 0 − Slave active, master inactive. Therefore outputs of the slave become Q = 0 and Q'= 1. Again clock = 1 − Master active, slave inactive. Therefore even with the changed outputs Q = 0 and Q'= 1 fed back to master, its output will be Q = 0 and Q' = 1. That means S = 0 and R = 1. Hence with clock = 0 and slave becoming active the outputs of slave will remain Q = 0 and Q' = 1. Thus we get a stable output from the Master slave.
3	**J = 1 and K = 0 (Set)**	Clock = 1 − Master active, slave inactive. Therefore outputs of the master become Q = 1 and Q'= 0. That means S = 1 and R =0. Clock = 0 − Slave active, master inactive. Therefore outputs of the slave become Q = 1 and Q'= 0. Again clock = 1 − then it can be shown that the outputs of the slave are stabilized to Q = 1 and Q' = 0.
4	**J = K = 1 (Toggle)**	Clock = 1 − Master active, slave inactive. Outputs of master will toggle. So S and R also will be inverted. Clock = 0 − Slave active, master inactive. Outputs of slave will toggle. These changed output are returned back to the master inputs. But since clock = 0, the master is still inactive. So it does not respond to these changed outputs. This avoids the multiple toggling which leads to the race around condition. The master slave flip flop will avoid the race around condition.

Registers

- Registers are groups of flip-flops (FF), where each flip-flop(FF) is capable of storing one bit of information.
- An *n*-bit register is a group of *n* flip-flops. The basic function of a register is to hold information in a digital system and make it available to the logic elements for the computing process.
- A register is a group of binary cells suitable for holding binary information.
- A group of cascaded flip flops used to store related bits of information is known as a register.
- A *register* is a very small amount of very fast *memory* that is built into the CPU (central processing unit) in order to speed up its operations by providing quick access to commonly used values.
- Flip-flop is a 1 bit memory cell which can be used for storing the digital data. To increase the storage capacity in terms of number of bits, we have to use a group of flip-flop. Such a group of flip-flop is known as a **Register**.
- The binary data in a register can be moved within the register from one flip-flop to another. The registers that allow such data transfers are called as **shift registers**.

There are four mode of operations of a shift register.

a) Serial Input Serial Output
b) Serial Input Parallel Output
c) Parallel Input Serial Output
d) Parallel Input Parallel Output



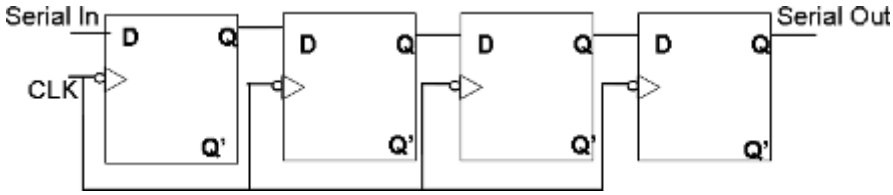
Application of Registers

These are used in computers for

- Temporary storage
- Data transferring
- Data manipulation
- As counters

Serial-In, Serial-Out (SISO)

Serial-In, Serial-Out shift register can be constructed by using D flip flops. This type of shift register accepts data serially i.e. one bit at a time and produces stored information on its output serially.



Here, four flip flops are cascaded. Since each flip flop can store only one bit, the register can store maximum four bits. More flip flops can be cascaded to store more than 4 bits. Clock is applied simultaneously to all flip flops clocking them synchronously.

We know that in a D Flip Flop, the Q output is identical to the D input except with one pulse time delay. Therefore, there will be a delay i.e. it will take one clock pulse to transfer the bit to next flip flop.

For example, consider that all the stages are reset and a logical input 1011 is applied at the serial input line connected to stage A. and see the bellow table that how the data shifted from one flip flop to other and finally get the output from D flip flop.

Operation of the Shift-right Register

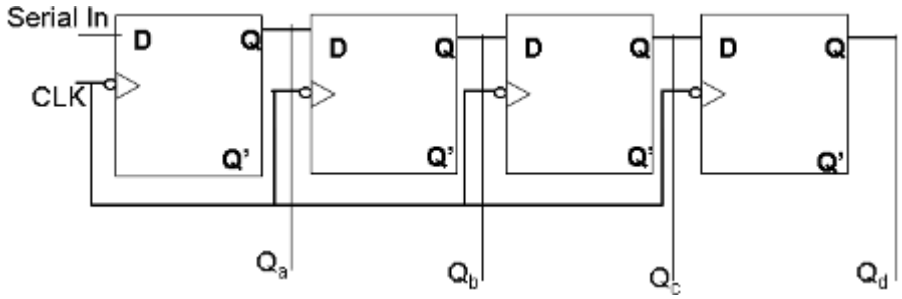
Timing pulse	Q_A	Q_B	Q_C	Q_D	*Serial output at* Q_D
Initial value	0	0	0	0	0
After 1st clock pulse	1	0	0	0	0
After 2nd clock pulse	1	1	0	0	0
After 3rd clock pulse	0	1	1	0	0
After 4th clock pulse	1	0	1	1	1

After fourth clock pulse we will get first input after next three clock pulse the complete input (1011) which we feed at flip flop A will out from flip flop D.

Serial-In, Parallel-Out (SIPO)

Serial-In, Parallel-Out shift register can be constructed by using D flip flops.

This type of shift register accepts data serially i.e. one bit at a time and produces stored information on its output parallely i.e. all the outputs are available simultaneously.



Here, four flip flops are cascaded. Since each flip flop can store only one bit, the register can store maximum four bits. More flip flops can be cascaded to store more than 4 bits. Clock is applied simultaneously to all flip flops clocking them synchronously.

We know that in a D Flip Flop, the Q output is identical to the D input except with one pulse time delay. Therefore, there will be a delay i.e. it will take one clock pulse to transfer the bit to next flip flop. All the outputs are available simultaneously at Q_a, Q_b, Q_c and Q_d.

Note: Circuit diagram of SIPO is same as SISO except that outputs are available simultaneously rather than serially at Serial Out.

Bidirectional Shift Register

Bidirectional shift registers are the storage devices which are capable of shifting the data either right or left depending on the mode selected.

It can be implemented using AND and OR logic gates circuitry that enables the transfer of data from one stage to the next stage to the right or to the left, depend on the level of control line.

The RIGHT/LEFT is the control input signal which allows data shifting either towards right or towards left.

A high on this line enables the shifting of data towards right and low enables it towards left.

When RIGHT/LEFT is high, gates 1,3,5,7 are enabled.

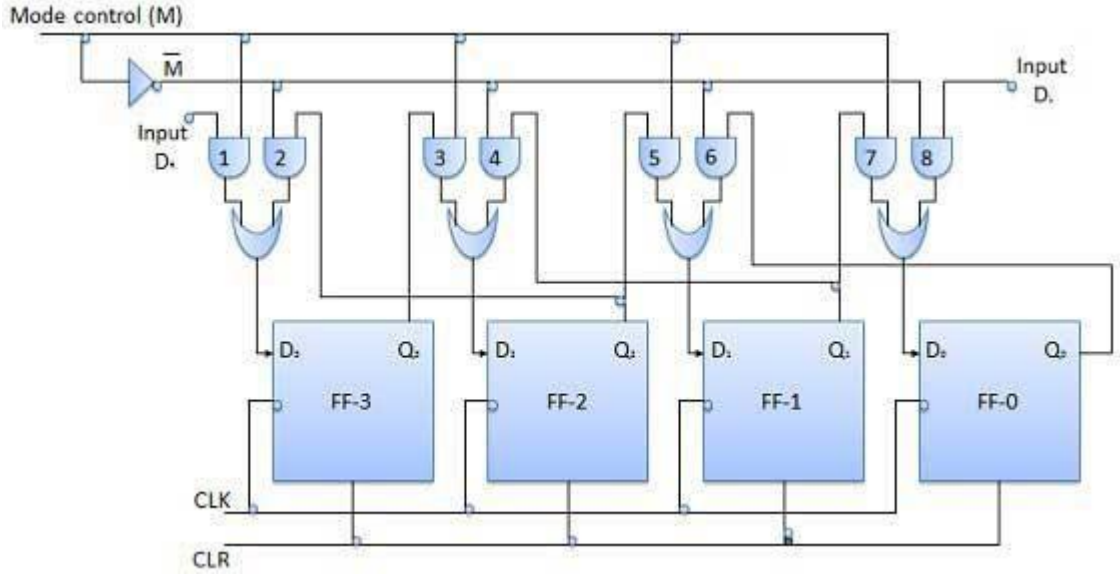
The state of Q output of each flip flop is passed through the D input of the following flip flop.

When the pulse arrives, the data are shifted one place to the right.

When the RIGHT/LEFT signal is low, gates 2,4,6,8 are enabled.

The Q output of each flip-flop is passed through the D input of the preceding flip-flop.

Block Diagram

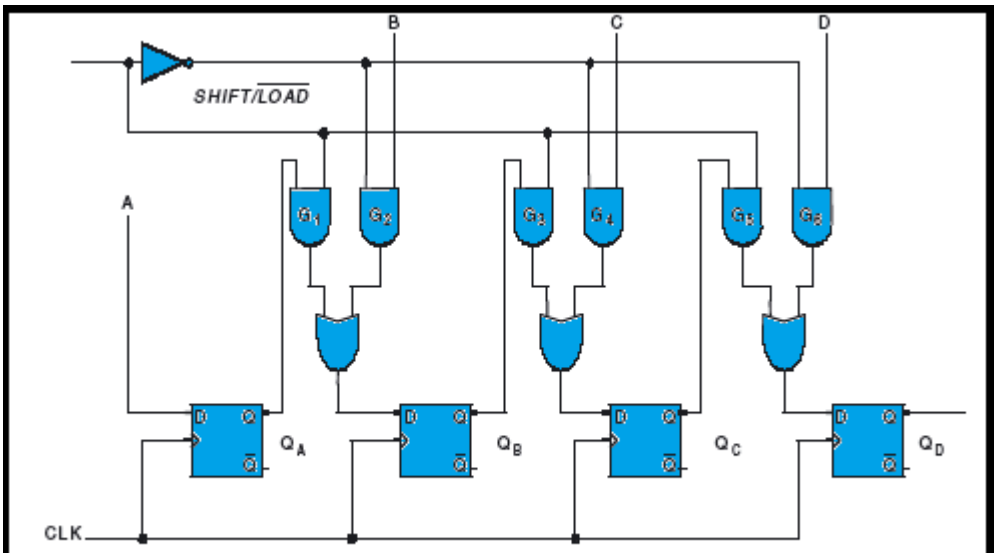


Operation

S.N.	Condition	Operation
1	**With M = 1 – Shift right operation**	If M = 1, then the AND gates 1, 3, 5 and 7 are enabled whereas the remaining AND gates 2, 4, 6 and 8 will be disabled. The data at D_R is shifted to right bit by bit from FF-3 to FF-0 on the application of clock pulses. Thus with M = 1 we get the serial right shift operation.
2	**With M = 0 – Shift left operation**	When the mode control M is connected to 0 then the AND gates 2, 4, 6 and 8 are enabled while 1, 3, 5 and 7 are disabled. The data at D_L is shifted left bit by bit from FF-0 to FF-3 on the application of clock pulses. Thus with M = 0 we get the serial right shift operation.

Parallel-In, Serial-Out (PISO)

Parallel-In shift register can be constructed by using D flip flops. This type of shift register accepts data parallely i.e. all the bits are input simultaneously.



A 4-bit parallel-in–serial-out shift register.

The input data will enter in parallel that means at a time to all flip flop. And output will get serially.

Suppose we have to save a 4-bit number (1011). Then all input are feed the inputs of different 4 number of flip flop. With single clock pulse all data are enter to all 4 **flip flops**.

A, B, C, and D are the four parallel data input lines and *SHIFT* / *LOAD* (*SH* / *LD*) is a control input that allows the four bits of data at A, B, C, and D inputs to enter into the register in parallel or shift the data in serial.

When *SHIFT* / *LOAD* is HIGH, AND gates G1, G3, and G5 are enabled, allowing the data bits to shift right from one stage to the next. When *SHIFT* / *LOAD* is LOW, AND gates G2, G4, and G6 are enabled, allowing the data bits at the parallel inputs.

When a clock pulse is applied, the flip-flops with D = 1 will be set and the flip-flops with D = 0 will be reset, thereby storing all the four bits simultaneously. The OR gates allow either the normal shifting operation or the parallel data-entry operation, depending on which of the AND gates are enabled by the level on the *SHIFT* / *LOAD* input.

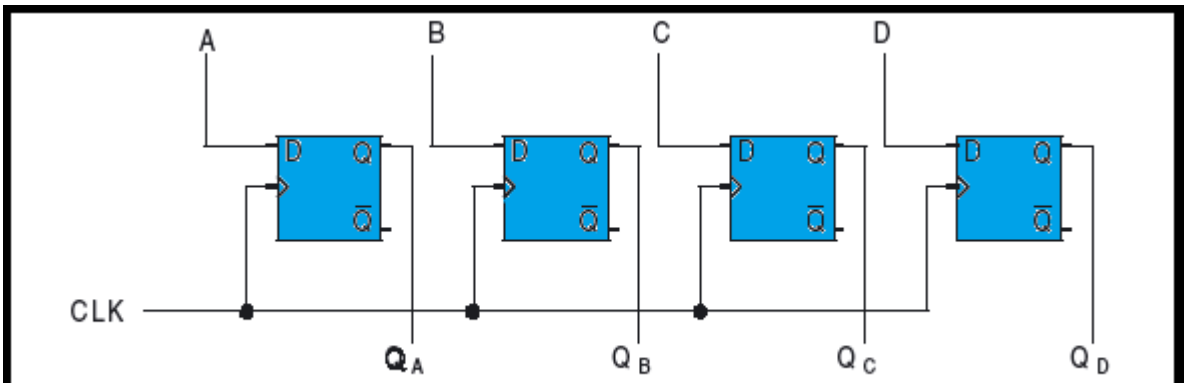
Parallel-In, Parallel-Out (PIPO)

Parallel-In shift register can be constructed by using D flip flops. This type of shift register accepts data parallely i.e. all the bits are input simultaneously and produces stored information on its output parallely i.e. all the outputs are available simultaneously.

Parallel in parallel out shift register, which is designed such that data can be shifted into or out of the register in parallel.

The data bits are entered into the flip flops simultaneously, rather than a bit-by-bit basis.

Let us take an example- Suppose we have to save a 4-bit number (1011). Then all input are feed the inputs of different 4 number of flip flop. With single clock pulse all data are enter to all 4 flip flops. Another very important thing in this type of register there is no interconnection between the flip flops since no serial shifting is required. Hence, the moment the parallel entry of the data is accomplished the data will be available at the parallel outputs of the register.



A 4-bit parallel-in–parallel-out shift register.

The parallel inputs to be applied at A, B, C, and D inputs are directly connected to the D inputs of the respective flip flops. On applying the clock transitions, these inputs are entered into the register and are immediately available at the outputs QA, QB, QC, and QD.

No clock pulse is required. Data can be shifted into or out of the register in parallel.

COUNTERS

A counter is a digital sequential logic device that will go through a certain predefined states (for example counting up or down) based on the application of the input pulses.

An electronic counter is used for counting the number of pulses coming at the input line in a specified time period. The counter must possess memory since it has to remember its past states.

The main purpose of counter is to record the number of occurrence of some input. Counter is the widest application of flip-flops. It is a group of flip-flops with a clock signal applied.

There are two main types of counters:

1) Asynchronous / Ripple Counter
2) Synchronous Counters

Asynchronous Counter

A ripple counter is an asynchronous counter where only the first flip-flop is clocked by an external clock. All subsequent flip-flops are clocked by the output of the preceding flip-flop.

In other words, in asynchronous counters, the CLK inputs of all flip-flops (except the first one) are triggered not by the incoming pulses but rather by the transition that occurs in other flip-flops. Therefore, the change of state of a particular flip-flop is dependent upon the present state of other flip-flops.

Asynchronous counters are also called ripple-counters because of the way the clock pulse ripples it way through the flip-flops.

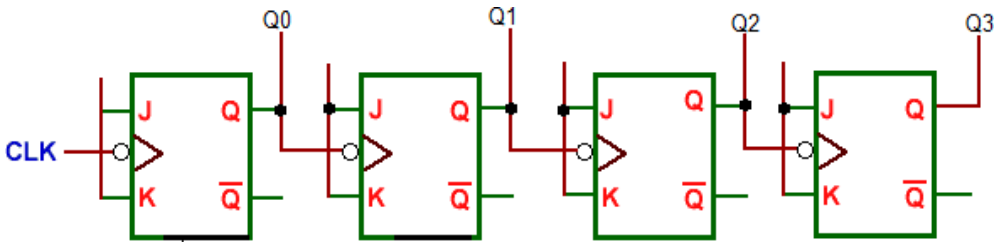
Different Types of Asynchronous Counters-

a) Asynchronous/Ripple Up Counter

b) Asynchronous/Ripple Down Counter

c) Bidirectional Counters(Up/Down)

a) Asynchronous/Ripple Up Counter



4-Bit Ripple Up counter.

A four-bit asynchronous counter is the external clock connected to the clock input of the first flip-flop (FF0) only.

FF0 changes state at the negative triggering of each clock pulse, but FF1 changes only when triggered by the negative triggering of the Q output of FF0. Because of the inherent propagation delay through a flip-flop, the transition of the input clock pulse and a transition of the Q output of FF0 can never occur at exactly the same time.

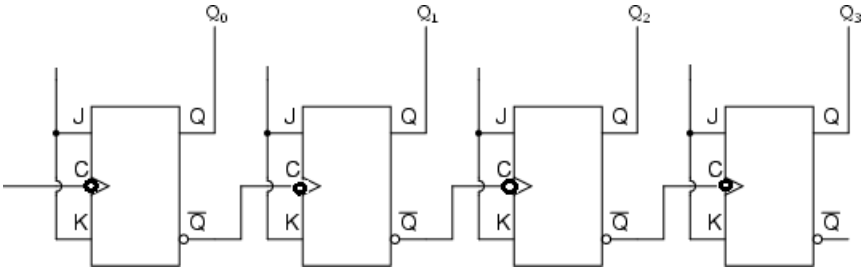
FF2 changes state at the negative triggering of the Q output of FF1, but FF3 changes only when triggered by the negative triggering of the Q output of FF2 and so on.

Therefore, the flip-flops cannot be triggered simultaneously, producing an asynchronous operation.

Counter Circuit Design Truth Table

Clock	Q3	Q2	Q1	Q0
1	0	0	0	0
2	0	0	0	1
3	0	0	1	0
4	0	0	1	1
5	0	1	0	0
6	0	1	0	1
7	0	1	1	0
8	0	1	1	1
9	1	0	0	0
10	1	0	0	1
11	1	0	1	0
12	1	0	1	1
13	1	1	0	0
14	1	1	0	1
15	1	1	1	0
16	1	1	1	1

b) 4-Bit Asynchronous/Ripple Down Counter



A 4 bit asynchronous DOWN is simple modification of the UP counter. 4 bit DOWN counter will count numbers from 15 to 0(1111 to 1000), downwards. The clock inputs of all flip flops are cascaded and the JK input of each flip flop is connected to logic 1.

That means the flip flops will toggle at each negative triggering of the clock signal. The clock input is connected to first flip flop. The other flip flops in counter receive the clock signal input from Q' output of previous flip flop, rather than Q output.

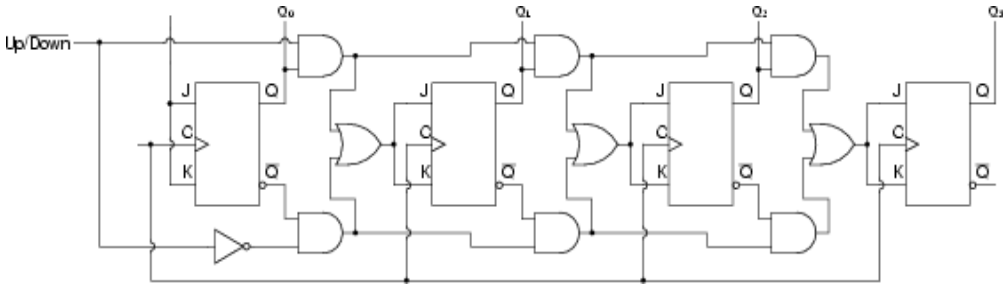
Here Q0, Q1, Q2, Q3 represents the count of the 4 bit down counter. The output of the first flip flop will change, when the negative triggering of clock signal occurs. For example, if the present count = 3, then the up counter will calculate the next count as 2. The input clock will cause the change in output (count) of the next flip-flop.

The operation of down counter is exactly opposite to the up counter operation. Here every clock pulse at the input will reduce the count of the individual flip flop. So the down counter counts from 15, 14, 13…0 i.e. (0 to 15) or 1111 to 0000.

<u>Counter Circuit Design Truth Table</u>

CK	Q_3	Q_2	Q_1	Q_0
0	1	1	1	1
1	1	1	1	0
2	1	1	0	1
3	1	1	0	0
4	1	0	1	1
5	1	0	1	0
6	1	0	0	1
7	1	0	0	0
8	0	1	1	1
9	0	1	1	0
10	0	1	0	1
11	0	1	0	0
12	0	0	1	1
13	0	0	1	0
14	0	0	0	1
15	0	0	0	0

c) Bi-directional Counter



Bi-directional Counter counts up or down depending on the status of the control signals UP and DOWN.

When the UP input is at 1 and the DOWN input is at 0, the AND network between FF0 and FF1 will gate the NOR output (Q) of FF0 into the clock input of FF1.

Similarly, Q of FF1 will be gated through the other AND network into the clock input of FF2. Thus the counter will count up.

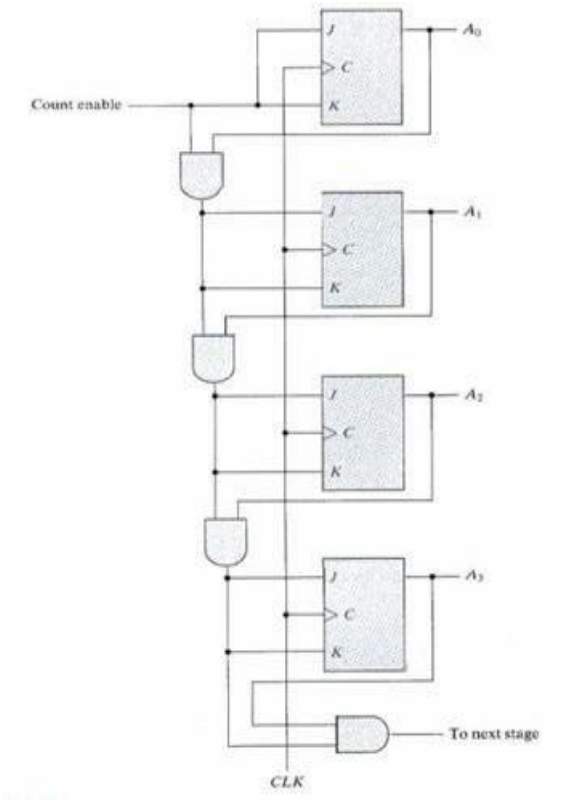
When the control input UP is at 0 and DOWN is at 1, the NOR outputs of FF0 and FF1 are gated into the clock inputs of FF1 and FF2 respectively.

Synchronous Counters

The counters which use clock signal to change their transition are called “Synchronous counters”. This means the synchronous counters depends on their clock input to change state values.

In synchronous counters, all flip flops are connected to the same clock signal and all flip flops will trigger at the same time.

Synchronous counters are also known as ‘ Simultaneous counters ’.There is no propagation delay and no ripple effect in synchronous counters.



4-bit Synchronous Counter

- The C inputs of all flip flops receive the common clocks. If the count enable input is 0,all J and K inputs are equal to “0” and the output of the counter does not change.

- The first stage A_0 is complemented when the counter is enabled and the clock goes through a positive transition.

- Each of the other three flip flops are complemented when all previous least significant flip flops are equal to 1 and the count is enabled.

- The chain of AND gates generate the required logic for the J & K inputs. The output carry can be used to extend the counter to more stages, with each stage having an additional flip flop and an AND gate.

- The inputs of first flip flop are connected to HIGH (logic 1), which makes the flip flop to toggle, for every clock pulse entered into it. So the synchronous counter will work with single clock signal and changes its state with each pulse.